Data Mastery 120: Roadmap Integral de Minería y Análisis de Datos

Día 1 – Conexión y Carga (Dataset: IBM HR)

Autor: Miguel Angel Rizo Vargas
Fecha: 24 de marzo de 2026

📌 Objetivo del Día 1

- **Conectar** el **dataset IBM HR** a tu **entorno SQL** (**PostgreSQL** + **DBeaver**).
- **Explorar** las primeras **filas** para **validar** la **carga**.
- **Construir** un **esquema ER inicial** del dataset.

🛠 Normalización de Columnas (snake_case)

Antes de cargar el **dataset** en **SQL**, **normalizamos** los **nombres** de las **columnas** para **evitar conflictos** de **mayúsculas/minúsculas**.

```
"""
TÍTULO: Pipeline de Normalización y Limpieza de Datos de RRHH
OBJETIVO: Automatizar la lectura de reportes de Recursos Humanos, estandarizar los
nombres de las columnas a formato 'snake_case' y exportar una versión limpia con
marca de tiempo, asegurando la compatibilidad con Excel y el orden histórico.
"""
# Herramientas para el manejo de datos y archivos CSV
import pandas as pd
import csv
# Utilidades para interactuar con las carpetas y rutas del sistema operativo
import os
from pathlib import Path
# Funciones para limpieza de texto (expresiones regulares) y gestión de fechas
import re
from datetime import datetime

# ---------------------------------------------------------
# 1. Configuración de rutas.
# ---------------------------------------------------------
# --- Configuración de rutas del proyecto ---
# Localiza la carpeta raíz subiendo tres niveles y define las rutas de entrada (raw)
# y salida (processed) para mantener los datos organizados.
base_dir = Path(__file__).resolve().parents[2]
target_raw_dir = base_dir / '02_data' / 'raw' / 'rrhh'
cleaned_target_dir = base_dir / '02_data' / 'processed' / 'rrhh'
# --- Identificación del archivo de origen ---
# Define el nombre específico del archivo de RRHH y construye su ruta
# completa para que el sistema pueda localizarlo.
```

```
rrhh_file_name = 'WA_Fn-UseC_-HR-Employee-Attrition'
rrhh_path_csv = os.path.join(target_raw_dir, f'{rrhh_file_name}.csv')
# --- Generación del archivo de salida con marca de tiempo ---
# Crea un nombre único para el archivo procesado usando la fecha y hora
actual.
# Esto evita que los resultados nuevos sobrescriban a los anteriores.
time_stamp = datetime.now().strftime('%Y%m%d_%H%M%S')
cleaned_file_name = f'rrhh_limpio_{time_stamp}.csv'
path_csv = os.path.join(cleaned_target_dir, cleaned_file_name)

# ---------------------------------------------------------
# 2. Normalización de Nombres de Columnas.
# ---------------------------------------------------------
def to_snake_case(name):
    """
    Convierte un texto de formato CamelCase o PascalCase a snake_case.
    """
    # Inserta un guion bajo entre una letra minúscula y una mayúscula que
inicia una palabra.
    # Ejemplo: "EmpleadoId" -> "Empleado_Id"
    s1 = re.sub('(.)([A-Z][a-z]+)', r'\1_\2', name)
    # Maneja casos con números o mayúsculas consecutivas y convierte todo
a minúsculas.
    # Ejemplo: "Empleado_Id" -> "empleado_id"
    return re.sub('([a-z0-9])([A-Z])', r'\1_\2', s1).lower()

# ---------------------------------------------------------
# 3. Carga del Datset.
# ---------------------------------------------------------
try:
    # --- Carga y normalización de datos ---
    # Importamos el archivo CSV a un formato de tabla (DataFrame) y
transformamos
    # todos los nombres de las columnas a 'snake_case' para estandarizar el
formato.
    df = pd.read_csv(rrhh_path_csv)
    df.columns = [to_snake_case(col) for col in df.columns]
    # --- Preparación para la exportación ---
    # Convertimos la tabla en una lista de diccionarios y extraemos los
nombres
    # de las columnas; esto facilita la escritura posterior del archivo limpio.
    datos_dict = df.to_dict(orient='records')
    mis_cabeceras = df.columns.tolist()
except Exception as e:
```

```
    # Gestión de errores en caso de que el archivo no exista o el formato sea incorrecto.
    print(f"❌ Error al cargar o procesar el dataset: {e}")
    datos_dict = None

# ---------------------------------------------------------
# 4. Exportación de Datos.
# ---------------------------------------------------------
def exportar_a_csv(ruta_destino, datos, cabeceras):
    """
    Gestiona la creación de carpetas y guarda los datos procesados en un archivo CSV.
    """
    if datos is None:
        return

    try:
        # --- Preparación del entorno ---
        # Verifica si existe la carpeta de destino; si no, la crea automáticamente
        # para evitar errores al intentar guardar el archivo.
        if not os.path.exists(cleaned_target_dir):
            os.makedirs(cleaned_target_dir, exist_ok=True)
            print(f'✅ Carpeta creada en: {cleaned_target_dir}')
        # --- Escritura del archivo ---
        # Abre el archivo con codificación 'utf-8-sig' para que Excel reconozca
        # correctamente tildes y eñes. Se usa el delimitador '|' para mayor claridad.
        with open(ruta_destino, mode='w', newline='', encoding='utf-8-sig') as archivo_csv:
            df.to_csv(archivo_csv, sep='|', index=False)
        # --- Confirmación y resumen ---
        # Muestra en consola un resumen técnico de los datos guardados y
        # confirma la ubicación final del archivo.
        print(df.info())
        print(f"\n✅ Proceso completado exitosamente.")
        print(f"🗁 Archivo guardado en: {ruta_destino}")
        print(f"🗎 CSV: {cleaned_file_name}")

    except PermissionError:
        # Error común si el archivo de destino está abierto en Excel o por otro usuario.
        print(f"❌ Error de Permiso: El archivo '{ruta_destino}' está abierto en otro programa.")
    except Exception as e:
```

```
        # Captura cualquier otro imprevisto durante la exportación.
        print(f"❌ Ocurrió un error inesperado al exportar: {e}")
# --- Punto de entrada del script ---
# Asegura que el proceso de limpieza y exportación solo se inicie si el
# archivo se ejecuta directamente (no si se importa desde otro script).
if __name__ == "__main__":
    print("--- Iniciando proceso de normalización ---\n")
    exportar_a_csv(path_csv, datos_dict, mis_cabeceras)
```

Ejemplo de columnas normalizadas:

- EmployeeNumber → employee_number
- Attrition → attrition
- JobRole → job_role
- MonthlyIncome → monthly_income
- PerformanceRating → performance_rating
- YearsAtCompany → years_at_company
- EducationField → education_field

🛠️ Guía Detallada Paso a Paso

📝 Paso 1. Instalación de PostgreSQL & DBeaver

- Descarga **PostgreSQL** desde el sitio oficial de **PostgreSQL** (https://www.postgresql.org/download/).
- Instala **DBeaver** para administrar la base (https://dbeaver.io/download/).

📝 Paso 2. Preparación del Entorno

Abrir **DBeaver** → Cliente gráfico para gestionar **PostgreSQL**.
Crear conexión a **PostgreSQL**:

- Menú: **Database** → **New Database Connection**.
- Selecciona **PostgreSQL**.
- Configura:
  - **Host**: localhost
  - **Port**: 5432
  - **Database**: postgres (por defecto)
  - User: postgres
  - Password: tu contraseña definida.
- Haz clic en **Test Connection** → debe mostrar **Success**.
- **Guarda** la conexión.

📝 Paso 3. Crear la Base de Datos ibm_hr

En el editor SQL de DBeaver:

- Abre una nueva consulta y ejecuta:

```
/*
==========================================================================
TÍTULO: INICIALIZACIÓN DEL ENTORNO DE DATOS - IBM HR ANALYTICS
    ==========================================================================

OBJETIVO:
Crear el contenedor principal (base de datos) para el almacenamiento de la
información de Recursos Humanos de IBM.

DESCRIPCIÓN DEL OBJETIVO:
Este es el primer paso del flujo de trabajo. Consiste en definir un espacio
lógico aislado y nombrado dentro del servidor de bases de datos, donde se
alojarán todas las tablas, dimensiones y vistas relacionadas con el análisis
de capital humano.
*/

-- Definición del contenedor de datos
-- Creamos la base de datos con un nombre descriptivo que indica el origen
(IBM)
-- y el propósito (HR Analytics).
CREATE DATABASE ibm_hr;

-- NOTA OPERATIVA: Una vez creada, asegúrate de activar la base de datos
-- (Set as default) para que todas las tablas posteriores se guarden en este
proyecto.
```

- Check de Ingeniería:
  - Confirmar que el nombre está en snake_case.
  - Validar que la base aparece en el panel izquierdo.

📝 Paso 4. Crear la Tabla Principal employee_master_data

Selecciona la base ibm_hr.

Ejecuta el script de creación de tabla con columnas normalizadas:

```
/*
==========================================================================
```

```
TÍTULO: CREACIÓN DE ESTRUCTURA PARA DATOS BRUTOS (RAW
DATA)
=========================================================
=======================

OBJETIVO:
Definir la tabla inicial "employee_master_data" para la ingesta masiva
de información
de Recursos Humanos.

DESCRIPCIÓN DEL OBJETIVO:
Esta tabla funciona como una "zona de aterrizaje" (Staging Area). Su
finalidad
es recibir los datos del archivo origen (CSV) sin procesar, manteniendo
todas
las métricas originales (desde salarios hasta niveles de satisfacción)
para
luego realizar la limpieza y normalización necesaria.
*/

CREATE TABLE IF NOT EXISTS employee_master_data(
-- Datos Demográficos y Personales
age INT NOT NULL,                    -- Edad del colaborador (obligatorio).
gender VARCHAR(50),                   -- Género registrado.
marital_status VARCHAR(20),           -- Estado civil (Single, Married,
Divorced).
over18 CHAR(1),                      -- Verificación de mayoría de edad (Y/N).

-- Indicadores de Retención y Movilidad
attrition VARCHAR(10),               -- Abandono de la empresa (Yes/No).
business_travel VARCHAR(50),          -- Frecuencia de viajes laborales.
distance_from_home INT,               -- Distancia al centro de trabajo en
km.

-- Estructura Organizacional y Puesto
department VARCHAR(50),               -- Área o departamento funcional.
job_role VARCHAR(50),                 -- Nombre del cargo o función.
job_level INT,                       -- Nivel de jerarquía en la organización.
employee_number INT PRIMARY KEY,      -- Identificador único (Llave
Primaria).
employee_count INT,                  -- Conteo unitario (usualmente 1 por
fila).

-- Compensación y Beneficios
monthly_income INT,                  -- Sueldo mensual bruto.
```

```
monthly_rate INT,                    -- Tasa o costo mensual relativo.
daily_rate INT,                      -- Pago por jornada diaria.
hourly_rate INT,                     -- Pago por hora laborada.
percent_salary_hike INT,             -- Porcentaje del último incremento.
stock_option_level INT,              -- Nivel de opciones sobre acciones.

-- Educación y Formación
education INT,                       -- Nivel académico alcanzado (codificado).
education_field VARCHAR(50),         -- Especialidad o carrera
estudiada.
training_times_last_year INT,        -- Cursos de capacitación tomados
el año pasado.

-- Encuestas de Satisfacción y Desempeño (Escalas numéricas)
environment_satisfaction INT,        -- Satisfacción con el entorno
laboral.
job_involvement INT,                 -- Nivel de compromiso con el puesto.
job_satisfaction INT,                -- Nivel de felicidad en el trabajo.
performance_rating INT,              -- Calificación de la última evaluación.
relationship_satisfaction INT,       -- Calidad de relación con colegas.
work_life_balance INT,               -- Equilibrio entre vida y trabajo.

-- Trayectoria y Permanencia
num_companies_worked INT,            -- Cantidad de empresas previas.
total_working_years INT,             -- Años totales de experiencia laboral.
years_at_company INT,                -- Antigüedad en la empresa actual.
years_in_current_role INT,           -- Tiempo en el puesto actual.
years_since_last_promotion INT,      -- Años desde el último ascenso.
years_with_curr_manager INT,         -- Tiempo bajo el mando del jefe
actual.

-- Condiciones Laborales
over_time VARCHAR(10),               -- Realización de horas extra
(Yes/No).
standard_hours INT-- Horas base establecidas por contrato.
);
```

- **Pitfall común**: olvidar "**;**" al final o usar **nombres con espacios**.
- **Check de Ingeniería**: todos los nombres en **snake_case**, tipos de datos correctos.

📝 Paso 5. Importar el Dataset IBM HR (Bulk Load)

Antes de **normalizar**, necesitamos los **datos** dentro. Usaremos el **asistente de DBeaver** para mayor precisión:

Opción 1:

1. Menú: **Tools** → **Data Transfer** → **Import Data**.
2. Selecciona **archivo CSV** del dataset **IBM HR**.
3. Destino: **tabla employee_master_data**.
4. Configura **delimitador** , y **encabezados**.
5. **Ejecuta** la **importación**.

Opción 2:

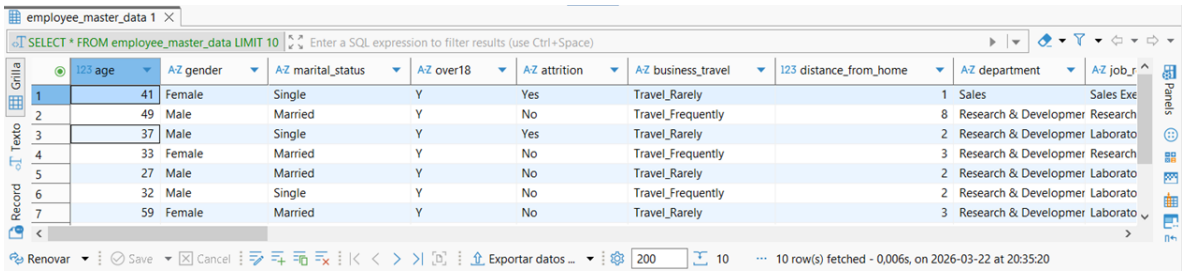
1. En **DBeaver**, haz clic derecho sobre la **tabla employee_master_data** creada ayer.
2. Selecciona "**Import Data**" -> **CSV**.
3. Busca tu **archivo de IBM HR**.
4. **Mapeo**: Asegúrate de que las **columnas** del CSV coincidan con los nombres de la tabla.
   Ejecutar: Una vez finalizado, **verifica** que los **datos** se **cargaron** correctamente.

📝 Paso 6. Validación de Datos

Confirma que los datos se cargaron correctamente:

```
/*****************************************************************************
******
Título: Visualización Preliminar de Datos de Empleados
Objetivo: Obtener una muestra rápida de los datos maestros de los
empleados.
Descripción: Esta consulta selecciona las primeras 10 filas de la tabla
        'employee_master_data' para verificar la estructura y el tipo de
        información contenida en ella.
*****************************************************************************
*****/

-- Selecciona todas las columnas y limita el resultado a 10 filas para
una vista rápida
SELECT*
```

```
FROMemployee_master_data
LIMIT10;
```



employee_master_data 1

SELECT * FROM employee_master_data LIMIT 10

	123 age	A-Z gender	A-Z marital_status	A-Z over18	A-Z attrition	A-Z business_travel	123 distance_from_home	A-Z department	A-Z job_r
1	41	Female	Single	Y	Yes	Travel_Rarely	1	Sales	Sales Exe
2	49	Male	Married	Y	No	Travel_Frequently	8	Research & Developmer	Research
3	37	Male	Single	Y	Yes	Travel_Rarely	2	Research & Developmer	Laborato
4	33	Female	Married	Y	No	Travel_Frequently	3	Research & Developmer	Research
5	27	Male	Married	Y	No	Travel_Rarely	2	Research & Developmer	Laborato
6	32	Male	Single	Y	No	Travel_Frequently	2	Research & Developmer	Laborato
7	59	Female	Married	Y	No	Travel_Rarely	3	Research & Developmer	Laborato

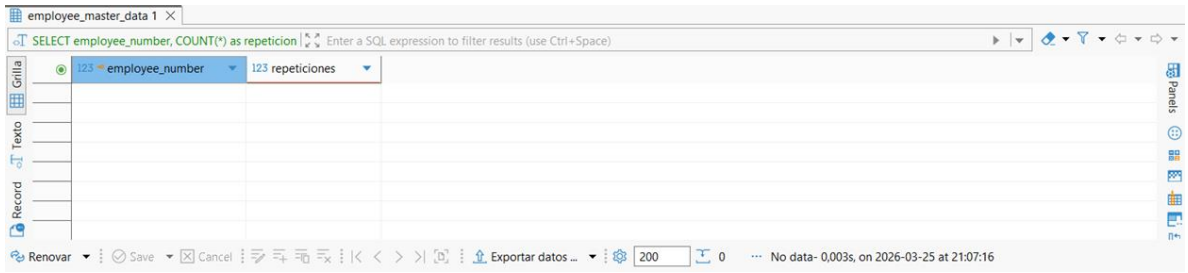
10 row(s) fetched - 0,006s, on 2026-03-22 at 20:35:20

📝 Paso 7. Explorar Datos Iniciales

- **Controles de la calidad de los datos (Data Quality Cheks)**:
  - **Detección de registros duplicados**:

```
/*
==========================================================
============================
TÍTULO: DETECCIÓN DE REGISTROS DUPLICADOS POR NÚMERO
DE EMPLEADO
==========================================================
============================

OBJETIVO:
Identificar si existen identificadores de empleado que se repiten
dentro de la
base de datos.

DESCRIPCIÓN DEL OBJETIVO:
En una base de datos de personal, el "Número de Empleado" debe
ser único. Esta
consulta agrupa la información por dicho identificador y filtra los
resultados
para mostrar solamente aquellos casos donde un mismo número
aparece más de una
vez, lo cual indicaría un error de carga o una duplicidad que debe
corregirse.
*/

-- Bloque de análisis de integridad de datos
SELECT
    employee_number,     -- El identificador único que estamos
    auditando
```

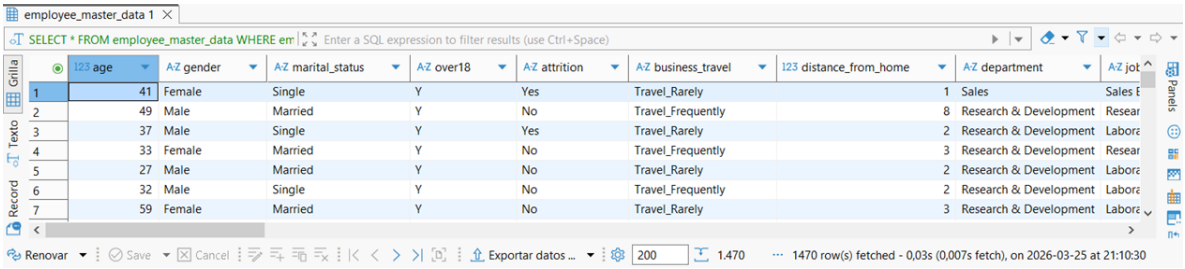
```
        COUNT(*) as repeticiones-- Cuenta cuántas veces aparece
        cada identificador
FROM
        employee_master_data-- La tabla principal que contiene los
        registros
GROUP BY
        employee_number-- Agrupa las filas que comparten el mismo
        número de empleado
HAVING
        COUNT(*) > 1;-- FILTRO CLAVE: Muestra solo los casos que
        se repiten (más de una vez)
```



- **Identificación de Valores Nulos en Columnas Críticas**:

```
/*
==========================================================
============================
TÍTULO: FILTRADO DE REGISTROS CON INFORMACIÓN ESENCIAL
PRESENTE
==========================================================
============================

OBJETIVO:
Recuperar únicamente las filas que contengan datos en al menos
uno de los campos
críticos de la tabla.

DESCRIPCIÓN DEL OBJETIVO:
Este script actúa como un primer filtro de calidad. Su finalidad es
extraer los
registros que no están completamente vacíos en sus columnas
clave (ID, área,
puesto y sueldo). Es útil para descartar filas "fantasma" o errores
de carga
donde se crearon registros sin contenido útil para el análisis de
nómina.
*/
```

```
-- Bloque de extracción y validación de contenido
SELECT*
FROM
    employee_master_data-- Tabla fuente con la información de
    los colaboradores
WHERE
    -- CRITERIOS DE VALIDACIÓN: El registro debe tener
    contenido en cualquiera de estos campos
    employee_number IS NOT NULL OR-- Que el ID de empleado
    no esté vacío
    department IS NOT NULL OR-- Que el departamento tenga un
    nombre asignado
    job_role IS NOT NULL OR-- Que el puesto de trabajo esté
    definido
    monthly_income IS NOT NULL;-- Que exista un registro del
    salario mensual
```

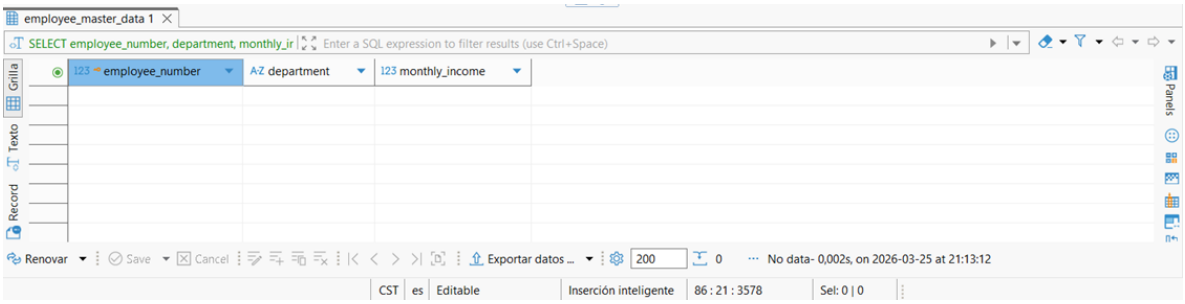


	age	gender	marital_status	over18	attrition	business_travel	distance_from_home	department	job
1	41	Female	Single	Y	Yes	Travel_Rarely	1	Sales	Sales E
2	49	Male	Married	Y	No	Travel_Frequently	8	Research & Development	Resear
3	37	Male	Single	Y	Yes	Travel_Rarely	2	Research & Development	Labora
4	33	Female	Married	Y	No	Travel_Frequently	3	Research & Development	Resear
5	27	Male	Married	Y	No	Travel_Rarely	2	Research & Development	Labora
6	32	Male	Single	Y	No	Travel_Frequently	2	Research & Development	Labora
7	59	Female	Married	Y	No	Travel_Rarely	3	Research & Development	Labora

- **Auditoría de Integridad (Valores fuera de rango o vacíos)**:

```
/*
==========================================================
============================
TÍTULO: IDENTIFICACIÓN DE REGISTROS CON DATOS FALTANTES
O ERRÓNEOS
==========================================================
============================

OBJETIVO:
Localizar empleados que tengan el departamento vacío o ingresos
mensuales
iguales o menores a cero.

DESCRIPCIÓN DEL OBJETIVO:
Este script sirve como una auditoría de integridad para la nómina.
Busca
```

```
detectar dos errores comunes: registros donde no se asignó un área de trabajo
(campo vacío) y registros con salarios inválidos o en cero, los cuales podrían
afectar los cálculos de promedios y presupuestos de la empresa.
*/

-- Bloque de selección de datos críticos para auditoría
SELECT
    employee_number, -- Identificador único del colaborador
    department,      -- Área asignada (buscando espacios en blanco)
    monthly_income-- Sueldo registrado (buscando valores no válidos)
FROM
    employee_master_data-- Tabla principal de recursos humanos
WHERE
    -- CRITERIOS DE ERROR:
    department = '' OR-- Filtra si el departamento está vacío (texto sin contenido)
    monthly_income <= 0;-- Filtra si el salario es cero o negativo (valor ilógico)
```

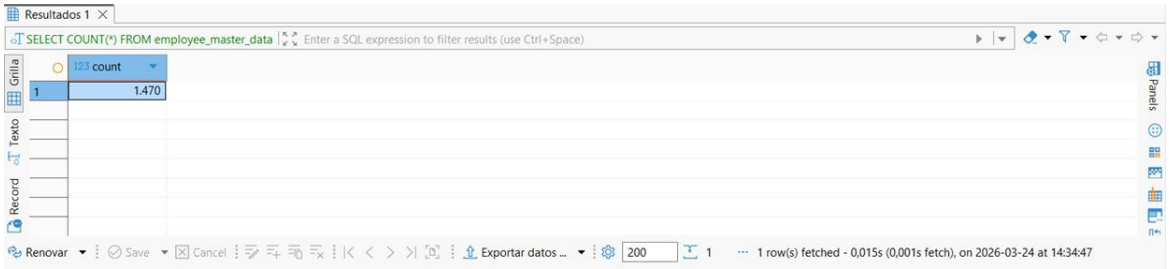


- **Ejecuta queries simples**:

```
/*
==========================================================================
TÍTULO: CONTEO TOTAL DE COLABORADORES EN LA TABLA MAESTRA
==========================================================================

OBJETIVO:
Obtener la cantidad exacta de registros almacenados en la tabla principal.
```

DESCRIPCIÓN DEL OBJETIVO:
Esta consulta se utiliza para validar el volumen de datos actual en la tabla
"employee_master_data". Es útil para confirmar que la carga de información
fue exitosa o para conocer el tamaño total de la plantilla activa e histórica.
*/

```
-- Bloque de consulta de métricas generales
SELECT
COUNT(*) -- Cuenta todas las filas individuales sin importar el contenido
FROM
employee_master_data;-- Tabla principal que contiene el censo de empleado
```

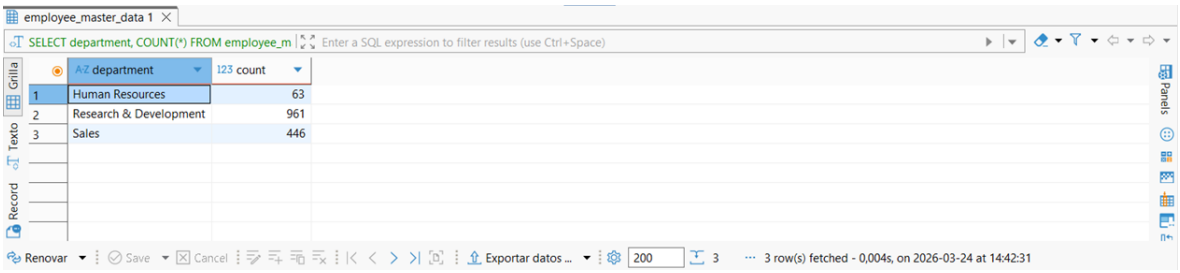


/*
==========
TÍTULO: DISTRIBUCIÓN DE PLANTILLA POR DEPARTAMENTO
==========

OBJETIVO:
Contabilizar cuántos empleados pertenecen a cada área de la organización.

DESCRIPCIÓN DEL OBJETIVO:
Esta consulta genera un resumen cuantitativo que permite visualizar el tamaño
relativo de cada departamento. Es fundamental para entender la estructura de
la empresa y detectar qué áreas concentran la mayor cantidad de capital humano.

```
*/

-- Bloque de selección y agrupación
SELECT
    department, -- Identifica el nombre del área o departamento
    COUNT(*)    -- Cuenta el total de empleados asociados a esa área
FROM employee_master_data-- Tabla principal de donde se extrae la información
GROUP BY department;-- Agrupa los resultados para que el conteo no sea global, sino por cada área
```



- **Pitfall común**: olvidar **GROUP BY** cuando usas agregaciones.
- **Check de Ingeniería**: validar que los resultados coinciden con expectativas (ej. número total de empleados).

📊 Paso 8. Construir Esquema ER (Entity-Relationship) en DBeaver

En **DBeaver**:

Opción 1:

- Vaya al **Explorador de proyectos** (normalmente a la izquierda).
- Haga **clic derecho** en la carpeta **Diagramas** (Diagrams).
- Seleccione **Crear nuevo diagrama**.
- Seleccionar **Base de Datos**: Póngale un **nombre** al diagrama (ej. **ibm_hr_erd**) y busque la base de datos **ibm_hr**.
- Selecciona la tabla **employee_master_data**.
- Clic en **Finalizar**.
- También puede **arrastrar y soltar tablas** directamente desde el navegador de bases de datos hacia el **lienzo** del diagrama.
- **Generación Automática**: DBeaver visualizará automáticamente las **relaciones** existentes basadas en las **claves foráneas (FK)** y **primarias (PK)**.
- **Exporta** como **imagen** o **PDF** para tu portafolio.

Opción 2:

- Vaya al **Navegador de Base de Datos** (normalmente a la izquierda).
- Busque la base de datos **ibm_hr**.
- Doble Clic en la tabla **employee_master_data**.
- Seleccione la carpeta **Diagramas** (Diagrams).

- **Blueprint lógico**: piensa en el **ER** como el **plano arquitectónico** del edificio de datos. Cada **columna** es un **pilar** que soporta **análisis futuros**.

📝 Paso 9. Conexión Python-SQL

Integra **SQL** con **Python** para extracción inicial:

```
"""
TÍTULO: Extracción Modular de Datos desde PostgreSQL
OBJETIVO: Consultar la tabla maestra de empleados de forma segura y organizada.
```

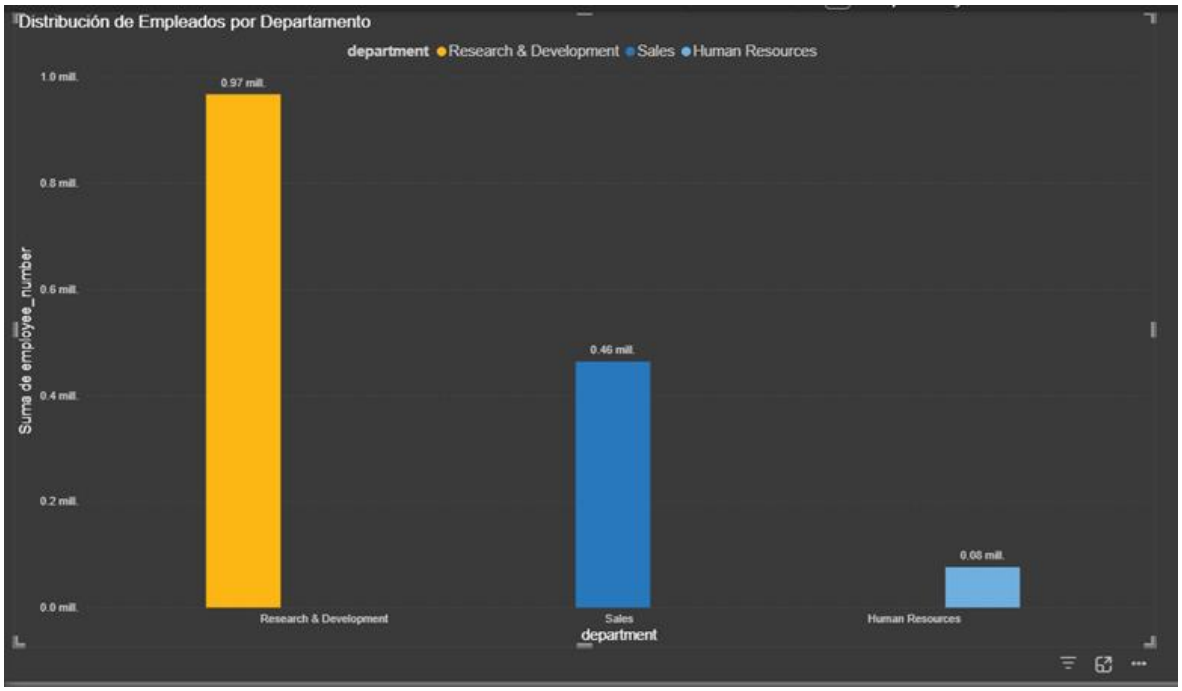
```
DESCRIPCIÓN: El script utiliza variables de entorno para proteger las
credenciales,
se conecta a la base de datos 'ibm_hr', extrae una muestra de
empleados y
gestiona automáticamente el cierre de la conexión para optimizar
recursos.
"""

import os
import psycopg2  # Conector para comunicarse con PostgreSQL
import pandas as pd  # Herramienta para analizar y organizar datos en
tablas
from dotenv import load_dotenv  # Carga variables secretas desde un
archivo .env

# Bloque de Configuración: Cargamos las credenciales ocultas del
sistema
load_dotenv()

def obtener_conexion_db():
    """
    Establece el puente de comunicación con el servidor.
    Usa 'os.getenv' para leer los datos de acceso sin exponerlos en el
código.
    """
    return psycopg2.connect(
        user=os.getenv('DB_USER'),
        password=os.getenv('DB_PASSWORD'),
        host=os.getenv('DB_HOST'),
        port=os.getenv('DB_PORT'),
        database=os.getenv('DB_NAME')
    )

# Definimos cuántos registros queremos traer por defecto
limite = 10

def extraer_datos_empleados(limite):
    """
    Lógica principal para realizar la consulta y traer los datos a Python.
    """
    conn = None
    try:
        # Iniciamos la conexión llamando a la función anterior
        conn = obtener_conexion_db()
```

```python
        # Definimos la instrucción SQL usando el límite solicitado
        query = f"""
        SELECT *
        FROM employee_master_data
        LIMIT {limite}
        """
        print("\n🚀 Iniciando ingesta PostgreSQL...")

        # pandas ejecuta la consulta y transforma el resultado en una tabla (DataFrame)
        df = pd.read_sql(query, conn)

        print("✅ ¡Datos cargados exitosamente!")
        return df

    except Exception as e:
        # Si algo falla (servidor caído, error de red), mostramos el motivo exacto
        print(f"❌ Error al conectar o procesar la base de datos: {e}")
        return None

    finally:
        # Bloque de Cierre: Pase lo que pase, nos aseguramos de cerrar la conexión
        if conn:
            conn.close()

# Bloque de Ejecución: Punto de partida del programa
if __name__ == "__main__":
    # Llamamos a la función principal usando el límite definido arriba
    df_empleados = extraer_datos_empleados(limite)

    # Si la descarga fue exitosa, mostramos los resultados en pantalla
    if df_empleados is not None:
        print("\n Vista previa de los datos:")
        print(df_empleados.head())
```

```
PROBLEMAS   SALIDA   CONSOLA DE DEPURACIÓN   TERMINAL   PUERTOS   HISTORIAL SQL   SUPERVISIÓN DE TAREAS   ···   Python

🚀 Iniciando ingesta PostgreSQL...
✅ ¡Datos cargados exitosamente!

   age  gender marital_status over18 attrition  ... years_in_current_role  years_since_last_promotion years_with_curr_manager over_time  standard_hours
0   41  Female        Single      Y       Yes  ...                    4                           0                       5       Yes              80
1   49    Male       Married      Y        No  ...                    7                           1                       7        No              80
2   37    Male        Single      Y       Yes  ...                    0                           0                       0       Yes              80
3   33  Female       Married      Y        No  ...                    7                           3                       0       Yes              80
4   27    Male       Married      Y        No  ...                    2                           2                       2        No              80

[5 rows x 35 columns]
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code>

Lín. 22, col. 1   Espacios: 4   UTF-8   CRLF   {} Python   Python 3.14.3
```

📝 Paso 10. Conexión Power BI-SQL

- **Abrir Power BI Desktop**:
  - Localiza el ícono de **Power BI Desktop** en tu menú de inicio o barra de búsqueda.
  - Haz clic para abrir la aplicación.
  - Espera a que cargue la pantalla inicial con la opción de **Obtener datos**.
- **Seleccionar Fuente de Datos → PostreSQL/SQL Server**:
  - En la pantalla principal, haz clic en **Obtener datos** (Get Data).
  - Se abrirá una ventana con múltiples opciones de fuentes.
  - El la opción **Base de Datos** selecciona **PostgreSQL**.
  - Haz clic en **Conectar**.
- **Configurar Servidor**:
  - En el cuadro de diálogo que aparece:
    - **Servidor**: escribe localhost (esto indica que la base está en tu propia máquina).
    - **Base de datos**: escribe **ibm_hr**.
  - Haz clic en **Aceptar**.
- **Autenticación**:
  - **Usuario**: postgres
  - **Contraseña**: la que definiste al instalar **PostgreSQL** (ej. SaPassword2026!).
  - Haz clic en **Conectar**.
- **Importar la Tabla**:
  - **Power BI** mostrará un navegador con las tablas disponibles en la base **ibm_hr**.
  - Selecciona la tabla que creaste (ej. employee_master_data).
  - Haz clic en **Cargar**.
  - **Power BI** importará los datos y los mostrará en el panel de campos.
- **Crear un Gráfico de Barras: Empleados por Department**:
  - En la vista de **Informe** (Report View), haz clic en el lienzo en blanco.
  - En el panel de **Visualizaciones**, selecciona el ícono de **Gráfico de barras agrupadas**.
  - Arrastra el campo department al eje **Categorías (Eje X)**.
  - Arrastra el campo employee_number al eje **Valores (Eje Y)**.
    - **Power BI** automáticamente contará el número de empleados por departamento.
  - **Ajusta el formato**:
    - **Título**: “Distribución de Empleados por Departamento”.
    - **Colores**: selecciona paleta corporativa o estándar.
    - **Etiquetas de datos**: activa para mostrar los valores sobre cada barra.



- Insights:
  - Predominio del Core de Innovación: Con 961 empleados, el área de Research & Development (R&D) es el core de innovación y ventaja competitiva. Operativamente, gestionar la retención de talento aquí es crítico, ya que cualquier cambio en este grupo de trabajo supone el mayor impacto financiero para la empresa.
  - Motor Comercial Consolidado: El área comercial (446 personas) sostiene el crecimiento escalable de Research & Development (R&D), actuando como un go-to-market consolidado que garantiza ingresos estables.
  - Estructura Administrativa Esbelta: El departamento de HH.RR. destaca por su estructura esbelta (63 profesionales), gestionando una ratio aproximada de 1:23. Esto refleja eficiencia administrativa, aunque sugiere implementar automatización para evitar la saturación del equipo.

- Validación del Paso 7:
  - SQL: confirmas que la tabla integral está disponible en la base ibm_hr.
  - Power BI: logras importar la tabla y visualizar un gráfico de barras.
  - Resultado esperado: un dashboard inicial que muestra la cantidad de empleados por departamento.